Journal of Network and Computer Applications 251 (2026) 104494



Contents lists available at ScienceDirect

Journal of Network and Computer Applications

journal homepage: www.elsevier.com/locate/jnca



Research paper



Forecast-driven task offloading for reliable and adaptive mobile edge computing

Ting Li, Yinan Mi, Kai Yang *

School of Computer, Electronics and Information, Guangxi University, Nanning, 530004, China

ARTICLE INFO

Keywords:
Mobile edge computing
Task offloading
TimesNet
LinUCB
Forecast-driven optimization

ABSTRACT

Mobile edge computing (MEC) is critical for latency-sensitive applications operating under highly dynamic workloads and reliability fluctuations. Existing optimization-based methods depend on static models, while learning-based methods remain reactive and lack foresight into future load or failure conditions. Although recent prediction-based methods introduce limited predictive signals into offloading decisions, their simple or task-specific predictors and complex decision architectures prevent them from capturing multi-scale temporal patterns or supporting real-time responsiveness. This paper proposes a forecast-driven task offloading framework that closes the loop between short-horizon forecasted modeling and online decision optimization. The framework employs TimesNet as a multi-step time-series learner to extract multi-period temporal structures from server load and failure dynamics, producing richer and more reliable forward-looking context than conventional pointwise forecasting. These forecasted results are then incorporated into an online offloading policy based on LinUCB (Linear Upper Confidence Bound), enabling lightweight, context-aware, and rapidly adaptive task offloading with sublinear regret while fully exploiting forecasted future conditions. By explicitly coupling forecasting with online learning, the proposed design enables proactive and reliability-aware offloading under nonstationary MEC conditions. Extensive experiments on real-world datasets show that our method reduces task failures and system cost while achieving faster convergence than state-of-the-art baselines. These results confirm the advantages of coupling multi-period forecasting with lightweight online decision-making for reliable and adaptive MEC systems.

1. Introduction

With the rapid proliferation of data-intensive and latency-critical applications such as autonomous driving (Kong et al., 2025), augmented reality (Xiu and Gorlatova, 2025), and services powered by large language models (LLM) (Siino et al., 2025), mobile edge computing (MEC) (Darchini-Tabrizi et al., 2025) has emerged as a pivotal paradigm that extends cloud intelligence to the network edge. By offloading computation tasks from resource constrained user devices (UDs) to nearby edge servers (ESs), the platform reduces end-to-end latency and energy consumption and enables real time service delivery (Song et al., 2025). As the number of UDs and ESs increases significantly, however, time-varying wireless links (Tian et al., 2025), fluctuating workloads (Esmaeili et al., 2025), and intermittent failures of ESs (Yang et al., 2025) introduce new challenges for reliable task execution. A recent survey emphasizes that offloading strategies must account for nonstationary dynamics and reliability constraints to sustain service quality in practice (Dong et al., 2024a).

Existing research on task offloading in MEC can be broadly divided into two categories: optimization-based methods and learning-based methods. The optimization-based methods formulate interpretable mathematical models that jointly minimize latency, energy, and cost under service and resource constraints (Deng et al., 2024; He et al., 2025; Liang et al., 2025; Kim et al., 2024; He et al., 2024; Dong et al., 2024b). These formulations provide theoretical guarantees and system-level insights but suffer from model mismatch under distribution shifts and time-varying conditions. Learning-based methods, such as deep reinforcement learning and multi-armed bandits (Tang and Wong, 2020; Hsieh et al., 2023), enable adaptive decision-making from interaction data without explicit system modeling (Kong et al., 2023; Qin et al., 2025; Wang et al., 2020; Tang and Wong, 2020; Xie and Cui, 2025; Hsieh et al., 2023). However, these methods remain fundamentally reactive because they rely solely on instantaneous observations and do not exploit the short-term predictability of workload and reliability dynamics. This limitation reduces their ability to anticipate congestion and proactively mitigate failures. To improve adaptability and reduce

* Corresponding author.
E-mail addresses: liting0715@gxu.edu.cn (T. Li), 2413393046@st.gxu.edu.cn (Y. Mi), yangkai@gxu.edu.cn (K. Yang).

https://doi.org/10.1016/j.jnca.2026.104494
Received 3 December 2025; Received in revised form 5 March 2026; Accepted 19 April 2026
Available online 21 April 2026
1084-8045/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

initial performance degradation, recent decentralized offline-to-online reinforcement learning frameworks warm up models using historical logs and then adjust them online (Lin et al., 2024). Although these approaches enhance responsiveness across cooperative edge nodes, they still lack mechanisms to incorporate predictions of future server load or failure states, leaving their decisions confined to current or past conditions.

Recent studies have shown that incorporating predicting signals can substantially improve the efficiency and robustness of edge systems. Accurate predictions of workload intensity, queue states, and reliability conditions enable proactive offloading decisions that prevent overload induced instability and improve system utilization (Nie et al., 2025a). For example, in vehicular edge environments, spatiotemporal prediction combined with cache assisted task migration reduces end-to-end delay and alleviates backhaul pressure (Zhu et al., 2025). Diwaker et al. (2024) predicted CPU core utilization for adaptive task scheduling. Chhabra et al. (2025) designed an offloading strategy for vehicular edge environments that relies on performance prediction. Madiyev et al. (2025) combined predictive analysis with deep Q-learning to achieve energy efficient offloading. Recent studies have introduced predictive intelligence into edge computing systems. Diwaker et al. (2024) predicted CPU core utilization for adaptive scheduling, while Chhabra et al. (2025) and Madiyev et al. (2025) employed performance or workload predicting to guide task offloading. These works demonstrate the potential of prediction for improving adaptability, but most methods provide only single-step estimates and typically rely on conventional time-series models or task-specific predictors that struggle to capture the multi-scale temporal patterns and nonstationary dynamics inherent in MEC systems. Moreover, their decision modules often depend on heavy learning models, limiting real time responsiveness.

Motivated by these observations, this paper proposes a **forecast-driven task offloading framework** that closes the loop between forecasting and online decision optimization. The framework employs a multi-step time-series learner, i.e., TimesNet (Wu et al., 2023), to generate robust short-horizon forecasts by explicitly capturing multi-period temporal variations in server load and failure dynamics. Unlike pointwise prediction, such structured forecasting provides richer forward-looking signals, enabling the decision module to anticipate future conditions more accurately and support more proactive offloading. Moreover, these forecasted contexts are fed into an online decision module based on LinUCB (Linear Upper Confidence Bound) (Shustova et al., 2025), enabling lightweight, context-aware, and rapidly adaptive task offloading. By tightly coupling forecasting with online learning, the proposed forecast-to-decision framework enables more effective use of anticipated load and reliability trends, thereby improving system responsiveness and reliability in highly dynamic MEC environments. The main contributions of this work are summarized as follows:

- **A forecast-driven task offloading framework.** We present a unified forecast-to-decision framework for dynamic MEC environments. To the best of our knowledge, this work is among the first to tightly couple multi-step multi-period forecasting (TimesNet) with a lightweight contextual bandit (LinUCB) for proactive and reliability-aware MEC offloading. This design closes the loop between forecasting and offloading, enabling proactive, reliability-aware, and adaptive task scheduling under nonstationary system dynamics.
- **A TimesNet-based forecast module and a LinUCB-based task offloading strategy.** We introduce a multi-step time series forecasting module based on TimesNet that captures multi-period temporal variations in server load and failure dynamics, producing richer and more reliable predictive context than conventional predictors. These forecasted contexts are integrated into an online offloading strategy based on LinUCB, which maintains lightweight computation, supports context-aware adaptation, and achieves sublinear regret. The resulting forecast-to-decision mechanism allows the offloading policy to exploit anticipated workload and reliability trends more effectively than existing prediction-based approaches.
- **Extensive experimental evaluation.** We conduct comprehensive experiments on real-world multivariate traces and simulated failure processes. The proposed method achieves faster convergence, lower task failure rate, and reduced system cost compared with state-of-the-art optimization-based and learning-based baselines. Ablation studies further demonstrate the individual contributions of the forecasting module and the decision layer to both reliability and efficiency.

2. Related work

Task offloading is a core mechanism in MEC for satisfying stringent latency and reliability requirements under dynamic workloads and resource variability. As MEC systems grow more heterogeneous and time-varying, it is challenging to design offloading strategies that remain both efficient and adaptive. These challenges have motivated three major lines of research: optimization-based, learning-based, and prediction-enhanced approaches.

2.1. Optimization-based task offloading methods

Optimization-based task offloading methods formulate offloading as a constrained optimization problem, targeting latency, energy consumption, or resource costs under service and capacity constraints (Deng et al., 2024; He et al., 2025; Liang et al., 2025; Kim et al., 2024; He et al., 2024; Dong et al., 2024b). Representative techniques include convex relaxation, Lyapunov drift control, distributed dual decomposition, and queue stability frameworks (Deng et al., 2024; He et al., 2025; Dong et al., 2024b). For example, Deng et al. (2024) developed an offloading model that is aware of delay and energy, and that jointly optimizes computation and transmission decisions through convex relaxation, achieving near optimality under fixed system assumptions. He et al. (2025) applied Lyapunov drift control to ensure long-term queue stability while balancing latency and energy, demonstrating strong theoretical guarantees but limited adaptivity under rapid workload fluctuations. Dong et al. (2024b) further employed distributed dual decomposition to coordinate multi-user offloading across heterogeneous edge nodes, reducing computation overhead but relying on accurate parameterization of resource and channel models. Although these methods provide rigorous analytical structures, their performance depends heavily on accurate system modeling and static parameter assumptions, making them less suitable for highly dynamic and nonstationary MEC environments.

2.2. Learning-based task offloading methods

To improve adaptability, learning-based task offloading methods employ deep reinforcement learning (DRL) or contextual bandits to make data-driven decisions without explicit system modeling (Kong et al., 2023; Qin et al., 2025; Wang et al., 2020; Tang and Wong, 2020; Xie and Cui, 2025; Hsieh et al., 2023). For example, Tang and Wong (2020) applied deep Q-learning to jointly optimize computation offloading and resource allocation, demonstrating that DRL can reduce latency in dynamic MEC environments but requires substantial exploration overhead and long training cycles. Hsieh et al. (2023) further developed a multi-agent DRL framework for cooperative task assignment across distributed ESs, achieving improved load balancing while still relying on large volumes of interaction data. In addition, Kong et al. (2023) proposed AccuMO, which integrates DRL with fine-grained task characteristics for accuracy-centric offloading in mobile AR systems, showing strong performance but at the cost of high model complexity and limited real-time applicability. Nevertheless, despite improved adaptability, these learning-based frameworks remain fundamentally reactive because they rely solely on instantaneous system observations and lack predictive awareness of future workload and reliability dynamics. This limitation underscores the need for short-horizon prediction mechanisms to support proactive offloading under highly dynamic MEC conditions.

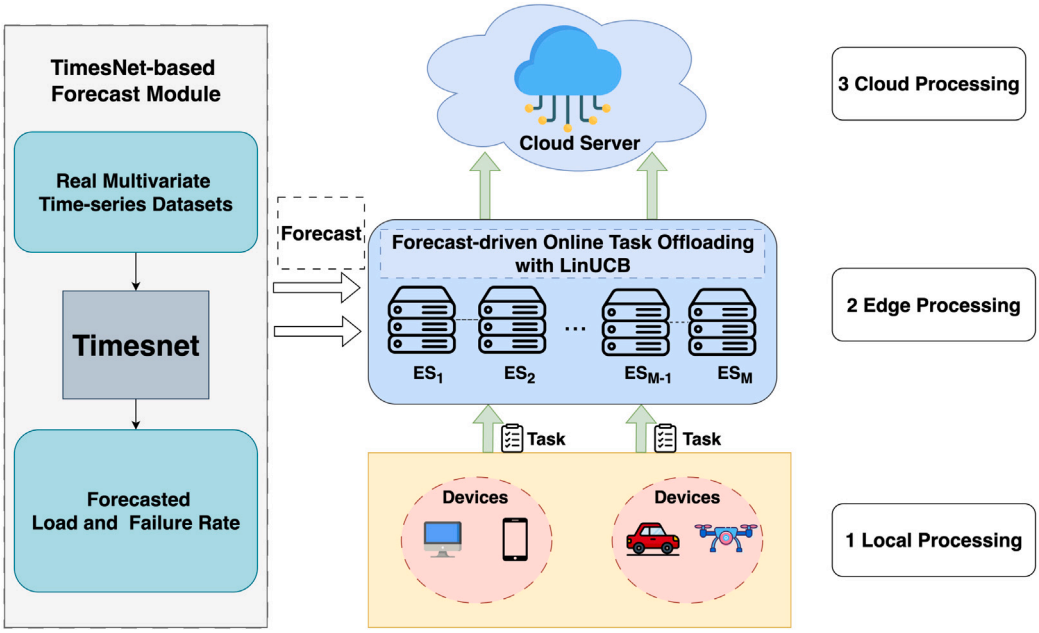


Fig. 1. Forecast-driven task offloading system model.

2.3. Prediction-based methods

Prediction-based task offloading methods incorporate predictive signals to support proactive scheduling in dynamic MEC environments. In vehicular edge systems, spatiotemporal prediction combined with cache-assisted task migration has been shown to reduce end-to-end delay and alleviate backhaul pressure (Zhu et al., 2025). Diwaker et al. (2024) proposed OCL-MEC, which employs an online regression model to estimate near-future CPU core utilization, enabling CPU core allocation before contention occurs. Chhabra et al. (2025) developed a performance-prediction-based offloading strategy that forecasts task arrival rates and computational demand to select suitable edge nodes under fast mobility. Madiyev et al. (2025) integrated predictive workload analysis into a deep Q-learning framework to improve task distribution across mobile, edge, and cloud layers. These studies collectively demonstrate the benefits of incorporating predictive information to enhance adaptability and mitigate overload.

Despite these advantages, existing prediction-based methods typically rely on simple or task-specific predictors that provide only pointwise or single-step estimates, limiting their ability to capture the multiscale temporal patterns and nonstationary dynamics inherent in MEC workloads. Furthermore, many of these approaches embed prediction signals into heavyweight decision architectures, which restricts real-time responsiveness and prevents full exploitation of predictive information during online decision making. **To address these limitations, we develop a forecast-driven task offloading framework that couples multi-period temporal learning via TimesNet with a lightweight LinUCB decision module, enabling efficient use of forward-looking information for real-time adaptive offloading.**

3. System model and problem formulation

3.1. System model

Fig. 1 illustrates the proposed forecast-driven task offloading framework, which operates in a three layer MEC architecture composed of the device layer, the edge layer, and the cloud layer. At the device layer, a set of heterogeneous UDs is denoted as $\mathcal{U} = \{U_1, U_2, \ldots, U_N\}$, generating computation intensive tasks represented by $D = \{D_1, D_2, \ldots, D_I\}$. For any user $U_n \in \mathcal{U}$, its generated task $D_i \in D$ can be executed locally or transmitted to the edge layer for offloading.

The edge layer consists of multiple edge servers $\mathcal{E} = \{ES_1, ES_2, \ldots, ES_M\}$, each reporting its workload and failure states over time. A TimesNet-based module captures temporal dependencies that span multiple periods from historical multivariate traces and outputs short-horizon forecasts for each server. For ES_m, the forecasted load and failure rate at time t are denoted as $\hat{L}_m(t)$ and $\hat{F}_m(t)$, and the forecasted context is $\hat{\mathbf{s}}_m(t) = \left(\hat{L}_m(t), \hat{F}_m(t)\right)$. This forecasted context is then passed to a LinUCB-based decision module, which determines the offloading destination. In experiments, the failure-rate context is produced by the TimesNet forecaster trained on real telemetry traces (SMD included), rather than being manually specified, and the resulting forecasted $\hat{F}_m(t)$ is the reliability-related input consumed by the LinUCB decision layer.

The offloading decision for task D_i is encoded as $\mathbf{X}_i = (X_i^l, X_i^e, X_i^c)$, where $X_i^l = 1$ indicates local execution, $X_i^e = 1$ indicates offloading to the edge, and $X_i^c = 1$ indicates offloading to the cloud. These variables satisfy $X_i^l + X_i^e + X_i^c = 1$, $X_i^l, X_i^e, X_i^c \in \{0, 1\}$. If task D_i is offloaded to the edge, the LinUCB module selects the optimal target server for offloading. The cloud layer provides supplementary computation and global coordination. By integrating forecast-based modeling with online contextual learning, the system forms a forecast-to-decision closed loop that enables proactive and reliable task scheduling in dynamic MEC environments (see Table 1).

3.2. Computation model

For each task D_i generated by user U_n, the execution delay depends on whether the task is processed locally, offloaded to an ES, or sent to the cloud.

3.2.1. Local computing model

When a task is executed locally ($X_i^l = 1$), the transmission delay and the computation delay are given by:

$$T_{loc}^{trans,i} = 0, \tag{1}$$

$$T_{loc}^{cal,i} = \frac{X_i^l \cdot D_i \cdot C_i}{f_l}. \tag{2}$$

Here X_i^l indicates that task D_i is executed locally, D_i denotes the input data volume of task i, C_i denotes the required CPU cycles per bit, and

Table 1
Notation of symbols used in the system model.

Notation	Meaning
X_i	The offloading decision variable of task i specifying its execution node
D_i	The input data volume of task i in bits
C_i	The CPU cycles required per bit for task i
T_i	The end-to-end execution latency of task i
f	The CPU operating frequency of the executing node
R_i	The achievable uplink data rate between the user device and edge server
P_i	The transmit power of the user device during offloading
h_i	The wireless channel gain from the user device to the edge server
B	The allocated wireless bandwidth for task offloading
N_0	The one sided power spectral density of channel noise
E	The total energy consumption of executing task i
BER_i	The bit error rate of the uplink channel for task i
θ	The weight of the reliability term in the optimization objective
$\hat{L}_m(t)$	The forecasted server load of edge server m at time t
$\hat{F}_m(t)$	The forecasted failure rate of edge server m at time t
$\mathcal{R}$	The overall reliability of successfully executing task i

f_l represents the local CPU frequency. The energy consumption of local execution is:

$$E_i^{loc} = k \cdot f_l^2 \cdot D_i \cdot C_i. \tag{3}$$

Here k is the hardware dependent energy coefficient.

3.2.2. Edge computing model

When a task is offloaded to an ES, transmission delay arises, and the achievable rate varies with wireless channel conditions. According to the Shannon Hartley theorem, the transmission rate R_i between the user devices (UDs) and the target ES is:

$$R_i = B \cdot \log_2\left(1 + \frac{P_i \cdot h_i}{N_0}\right). \tag{4}$$

Here B denotes the channel bandwidth, P_i is the transmission power of the device, h_i represents the channel gain, and N_0 denotes the noise power spectral density.

Using the rate R_i obtained from Eq. (4), the transmission and computation delays for executing task D_i at the edge can be expressed as:

$$T_i^{trans} = \frac{X_i^e \cdot D_i}{R_i}, \tag{5}$$

$$T_i^{cal} = \frac{X_i^e \cdot D_i \cdot C_i}{f_e}. \tag{6}$$

Here, X_i^e indicates that task D_i is offloaded to the ES, f_e represents the CPU frequency of the ES.

To incorporate the forecasted load obtained from the forecast module, let the short-horizon forecasted load for server ES_m at time t be $\hat{L}_m(t) \in [0, 1)$. We interpret the short-horizon forecasted load $\hat{L}_m(t) \in [0, 1)$ as the fraction of computing capacity already occupied at edge server ES_m, rather than a direct measurement of CPU frequency. Under a capacity-sharing approximation, the service rate available to newly arriving tasks scales with the remaining fraction of resources. Therefore, we adopt a conservative and monotone mapping

$$f_m^{\mathrm{eff}}(t) = f_e\big(1 - \hat{L}_m(t)\big), \tag{7}$$

where f_e denotes the nominal CPU frequency of ES_m, which represents the expected effective CPU service rate under forecasted contention. This linear form keeps the delay/energy model bounded, interpretable, while capturing the first-order impact of load on computing latency. Accordingly, the load-adjusted computation delay and energy consumption for task D_i when executed at ES_m are written as:

$$T_{\mathrm{i,edge}}^{\mathrm{cal}} = \frac{X_i^e\, D_i\, C_i}{f_m^{\mathrm{eff}}(t)} = \frac{X_i^e\, D_i\, C_i}{f_e\big(1 - \hat{L}_m(t)\big)}. \tag{8}$$

$$E_{\mathrm{i,edge}}^{\mathrm{cal}} = k_e\, f_e^2\big(1 - \hat{L}_m(t)\big)^2\, D_i\, C_i, \tag{9}$$

where k_e denotes the hardware-dependent energy coefficient. The total execution latency and energy consumption at the edge layer are given by:

$$T_{i,edge}^{total} = T_i^{trans} + T_{\mathrm{i,edge}}^{\mathrm{cal}}, \tag{10}$$

$$E_{i,edge}^{total} = E_i^{trans} + E_{\mathrm{i,edge}}^{\mathrm{cal}} = P_i \cdot T_i^{trans} + k_e \cdot f_e^2 \cdot \big(1 - \hat{L}_m(t)\big)^2 \cdot C_i \cdot D_i. \tag{11}$$

where P_i is the device's transmission power and k_e denotes the hardware related energy coefficient of the ES.

3.2.3. Cloud computing model

When a task is offloaded to the cloud ($X_i^c = 1$), the long physical distance between the device and the cloud results in a relatively large transmission delay. Let $r_{n,cloud}$ denote the uplink transmission rate from user U_n to the cloud. The cloud transmission delay is $T_{cloud}^{trans,i} = \frac{X_i^c \cdot D_i}{r_{n,cloud}}$. Moreover, due to the abundant computational resources in the cloud, the computation delay is negligible. Hence, the total delay in cloud can be described as $T_{cloud} = T_{cloud}^{trans,i}$.

3.2.4. Reliability model

Transmission reliability: In edge computing systems, wireless communication between UDs and ESs are vulnerable to interference, fading, and background noise, all of which may cause bit errors and packet corruption. To characterize the probability of error free transmission, an exponential error rate model is adopted. This exponential form yields a valid success probability in $(0, 1]$ and preserves monotonicity with respect to both task size and channel quality. Under the standard assumption of independent bit errors with instantaneous BER_i, the probability of error-free delivery over D_i bits is $(1 - BER_i)^{D_i}$, which is well-approximated by $\exp(-D_i BER_i)$ when BER_i is small. Accordingly, the transmission reliability of task D_i is expressed as:

$$\mathcal{R}_i^{trans} = e^{-\theta \cdot D_i \cdot BER_i}. \tag{12}$$

where θ is a tunable reliability sensitivity parameter, D_i represents the input data volume of task i, and BER_i denotes the instantaneous bit error rate of the wireless link. Larger task sizes or poorer channel quality naturally decrease the transmission reliability.

Execution reliability: Execution reliability reflects the probability that a task, once offloaded to an ES, can be successfully and correctly completed without interruption. In practical MEC environments, execution failures may arise from dynamic resource contention, queueing delays, thermal throttling, or transient hardware/software faults. As described in the system model, the forecast module provides short-horizon failure rate forecasts of each ES. Let the selected execution server for task D_i be ES_m, whose forecasted failure rate at time t is $\hat{F}_m(t)$. Using the exponential reliability formulation, we interpret $\hat{F}_m(t)$ as a failure-risk intensity over the decision horizon. Under a standard survival-model

view (memoryless risk accumulation), the success probability decays exponentially with the exposure. Here we use the required computation volume $C_i D_i$ as a proxy for execution exposure. the execution reliability of task D_i is given by:

$$\mathcal{R}_i^{exe} = e^{-\theta \cdot \hat{F}_m(t) \cdot C_i \cdot D_i}. \tag{13}$$

The total reliability of task D_i is defined as:

$$\mathcal{R}_i^{total} = \mathcal{R}_i^{trans} \cdot \mathcal{R}_i^{exe}. \tag{14}$$

In this expression, $\hat{F}_m(t)$ quantifies the likelihood of server side failures, while $C_i \cdot D_i$ represents the total computational demand of task i. The tasks with higher computation intensity or larger volume face a greater risk under unreliable servers. By integrating $\hat{F}_m(t)$, the execution reliability inherits the forecasting awareness provided by TimesNet and remains aligned with the forecast-to-decision loop. Assuming transmission and execution failures are conditionally independent given the current server state, we model the overall success probability multiplicatively, i.e., $\mathcal{R}_i^{total} = \mathcal{R}_i^{trans} \cdot \mathcal{R}_i^{exe}$.

3.3. Objective function

Based on the delay, energy consumption, and reliability models derived above, the system aims to jointly minimize the total execution latency and energy consumption while maximizing the overall task reliability. Let $T_i^{total} = X_i^l T_{loc}^{cal,i} + X_i^e T_{i,edge}^{total} + X_i^c T_{cloud}^{trans,i}$, $E_i^{total} = X_i^l E_i^{loc} + X_i^e E_{i,edge}^{total}$, and $\mathcal{R}_i^{total}$ denote the total delay, total energy consumption, and overall reliability of task D_i, respectively. The weighted optimization problem is formulated as follows:

$$\mathbf{P1}: \quad \min_{\{X_i^l, X_i^e, X_i^c\}} \sum_{i=1}^{I} \left(\alpha T_i^{total} + \beta E_i^{total} + \gamma (1 - \mathcal{R}_i^{total}) \right), \tag{15a}$$

$$\text{s.t.} \quad \text{C1: } X_i^l + X_i^e + X_i^c = 1, \quad \forall i, \tag{15b}$$

$$\text{C2: } X_i^l,\ X_i^e,\ X_i^c \in \{0,1\}, \quad \forall i, \tag{15c}$$

$$\text{C3: } \sum_{i=1}^{I} X_i^e \cdot \frac{D_i C_i}{\Delta t} \le \sum_{m=1}^{M} f_m^e \cdot \left(1 - \hat{L}_m(t)\right). \tag{15d}$$

To ensure numerical consistency when combining heterogeneous metrics, we treat α, β, and γ as scale-calibration coefficients. Since latency T_i^{total}, energy E_i^{total}, and the reliability penalty $(1-\mathcal{R}_i^{total})$ have different units and numeric ranges, we set $\alpha = 1.0$ as a reference and calibrate β and γ so that αT_i^{total}, βE_i^{total}, and $\gamma(1 - \mathcal{R}_i^{total})$ are of comparable order under typical operating conditions. Unless otherwise stated, we use $(\alpha, \beta, \gamma) = (1.0, 0.002, 40.0)$. The sensitivity study in Section 5.2.5 further confirms that the comparative trends remain stable under reasonable coefficient variations, indicating that the results are not an artifact of arbitrary numeric scaling. Eq. (15b) enforces that each task selects exactly one execution mode among local computing, edge offloading, and cloud processing. Eq. (15c) specifies that the decision variables X_i^l, X_i^e, and X_i^c are binary variables, indicating whether task D_i is assigned to the corresponding execution mode. Eq. (15d) imposes an aggregate computation capacity constraint on the edge layer, ensuring that the total required computation cycles from tasks offloaded to the edge do not exceed the forecasted available processing capability of all ESs, where the available capacity is adjusted according to the short-horizon load forecast $\hat{L}_m(t)$ from the forecast module.

3.4. Forecast error impact

The proposed offloading framework is driven by short-horizon forecasts of server load and failure dynamics. Since these predictive contexts are embedded into the computation, reliability, and capacity models, the forecasting errors can propagate to the system-level objective and influence the resulting offloading performance.

Load forecast impact: The forecasted load $\hat{L}_m(t)$ is mapped to the effective CPU frequency $f_m^{\text{eff}}(t) = f_e\left(1 - \hat{L}_m(t)\right)$ in Eq. (7) and directly enters the edge computation delay in Eq. (8). As a result, the computation latency becomes increasingly sensitive to the load estimate when the server utilization is high. Let $\hat{L}_m(t) = L_m(t) + \epsilon_L$ denote a perturbed load forecast. Under small perturbations, a first-order approximation yields

$$\frac{\Delta T_{i,\text{edge}}^{\text{cal}}}{T_{i,\text{edge}}^{\text{cal}}} \approx \frac{\epsilon_L}{1 - L_m(t)},$$

which indicates an error amplification effect as $L_m(t)$ approaches one. In addition, the edge capacity constraint in Eq. (15d) depends on the remaining capacity term $\left(1 - \hat{L}_m(t)\right)$. Therefore, a biased load forecast may shrink or expand the feasible region and change the set of admissible offloading actions.

Failure-rate forecast impact: The forecasted failure rate $\hat{F}_m(t)$ enters the execution reliability model in Eq. (13), i.e., $\mathcal{R}_i^{\text{exe}} = \exp\left(-\theta \hat{F}_m(t) C_i D_i\right)$, and the overall reliability is defined in Eq. (14). Let $\hat{F}_m(t) = F_m(t) + \epsilon_F$. Then the predicted execution reliability deviates from the true value by

$$\frac{\mathcal{R}_{\text{pred}}^{\text{exe}}}{\mathcal{R}_{\text{true}}^{\text{exe}}} = \exp\left(-\theta \epsilon_F C_i D_i\right),$$

which shows that the reliability term is exponentially sensitive to the failure-rate forecast error, especially for tasks with large computational demand.

In summary, forecasting errors affect offloading performance through two coupled mechanisms. The load forecast error distorts the effective service rate and tightens or relaxes the load-dependent capacity constraint, while the failure-rate forecast error perturbs the reliability term through the exponential mapping. These observations motivate the simulation-based sensitivity study reported in Section 5.2.5.

4. Algorithm design

As shown in Fig. 2, historical telemetry is fed to the TimesNet forecaster (offline training, online inference) to produce short-horizon forecasts $\hat{L}_m$ and $\hat{F}_m$. These forecasts are injected into the delay and reliability models (Eqs. (7) and (13)) to construct the forecast-driven context $x_{t,a}$ for each candidate action. LinUCB then selects the arm with the highest UCB score, executes the offloading action, observes the realized metrics/cost, and updates the selected-arm statistics, forming a closed decision loop.

4.1. TimesNet-based forecast module

Fig. 3 provides an overview of the TimesNet-based forecast module (Wu et al., 2023), which serves as the forecast engine for the proposed forecast-driven task offloading framework. The objective of this module is to generate short-horizon forecasts of server workload and failure rates, enabling the system to anticipate performance degradation before making offloading decisions. As illustrated in the figure, the module takes a window of recent multivariate telemetry as input and outputs short-horizon multi-step forecasts of server workload and failure dynamics, which are then injected into the delay, energy and reliability evaluation used by the downstream offloading policy. To better match a systems-paper style and highlight the forecast-to-decision contribution, we keep only the input–output definition and the integration interface in the main text, while moving the detailed TimesNet backbone formulations to Appendix A for completeness and reproducibility.

4.1.1. Input construction

To enable short-horizon forecast of workload and failure dynamics, each edge server ES_m continuously records a multivariate stream of system indicators such as server load, queue length, input traffic rate,

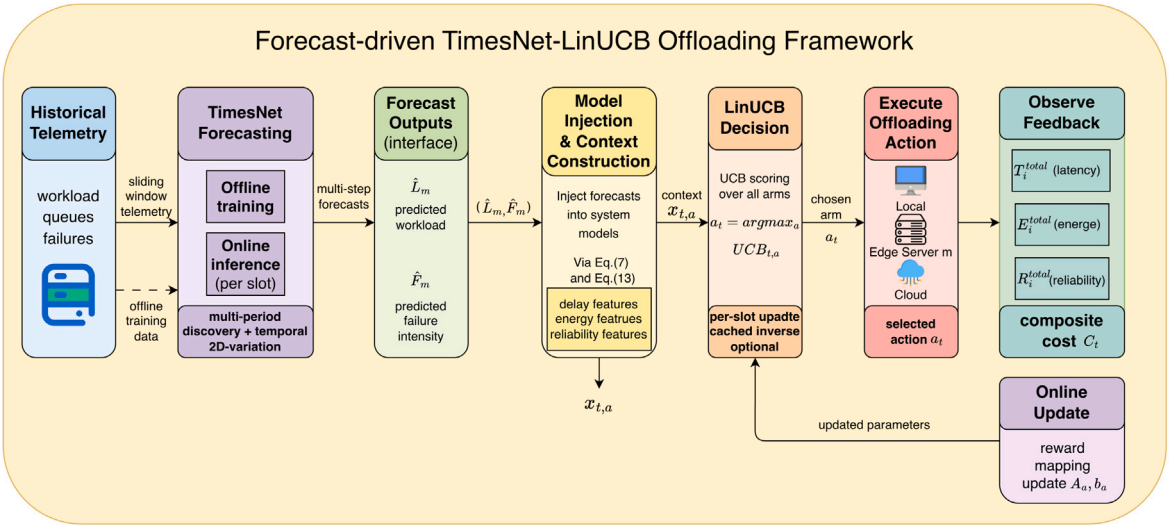


Fig. 2. Framework overview and information flow. TimesNet produces $\hat{L}_m$ and $\hat{F}_m$ to build $x_{t,a}$ for LinUCB.

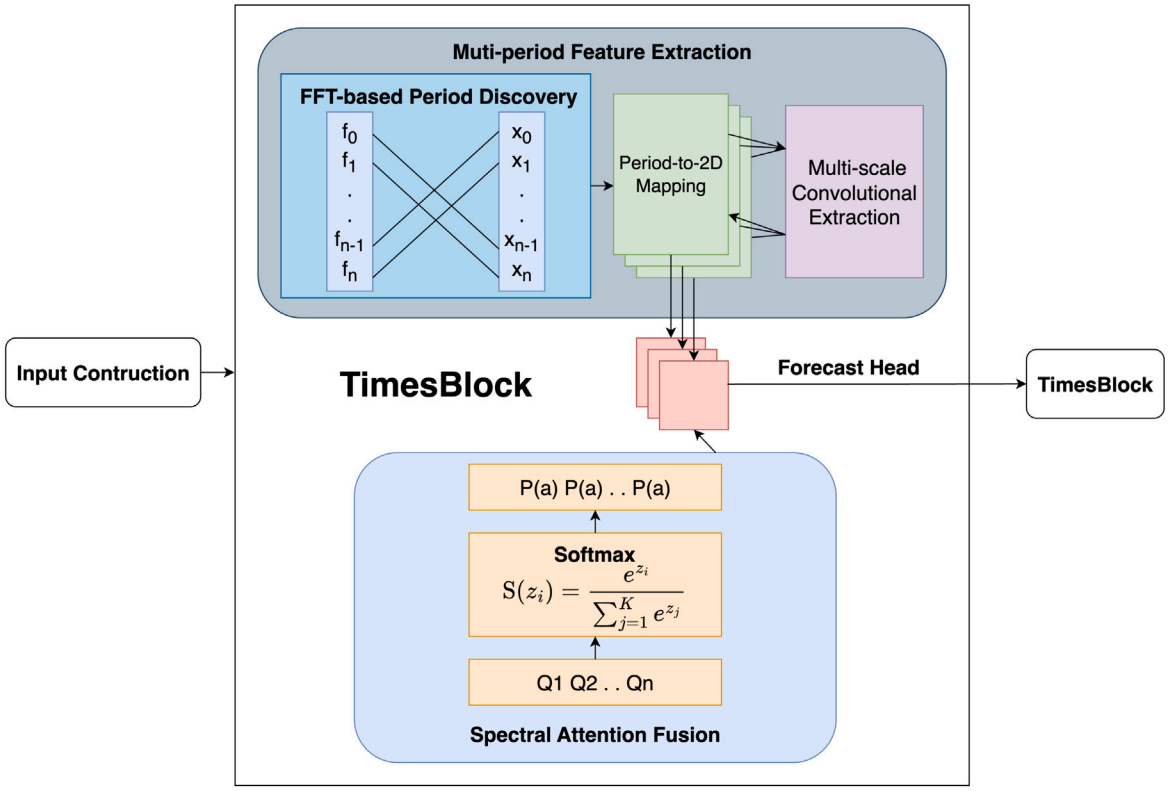


Fig. 3. Forecast-driven forecast module based on TimesNet.

and recent failure events. The failure-related indicator (constructed from telemetry/anomaly evidence) is included as one input channel, so the forecast head outputs both the workload forecast $\hat{L}_m(t)$ and the failure-rate forecast $\hat{F}_m(t)$ for downstream reliability-aware decision making. Let the C-dimensional state vector at time τ be denoted by $\mathbf{s}_\tau^{(m)} \in \mathbb{R}^C$. At each decision epoch t, the forecast module constructs a sliding window input by stacking the most recent L observations into a matrix:

$$\mathbf{X}_t^{(m)} = \left[\mathbf{s}_{t-L+1}^{(m)}, \ldots, \mathbf{s}_t^{(m)}\right] \in \mathbb{R}^{L\times C}. \tag{16}$$

This windowed representation captures both short term fluctuations and evolving temporal patterns in the server state. For each ES, the forecast objective is to forecast a sequence of future states over a horizon of length H. The forecasted target is defined as:

$$\mathbf{Y}_t^{(m)} = \left[\mathbf{y}_{t+1}^{(m)}, \ldots, \mathbf{y}_{t+H}^{(m)}\right] \in \mathbb{R}^{H\times C_{\text{pred}}}. \tag{17}$$

Here each vector $\mathbf{y}_{t+\tau}^{(m)}$ contains the normalized workload and failure rate values at time $t+\tau$. The TimesNet forecast module learns a parametric mapping between the input window and the forecast horizon:

$$\hat{\mathbf{Y}}_t^{(m)} = \mathcal{F}_{\text{TimesNet}}\left(\mathbf{X}_t^{(m)}; \Theta\right). \tag{18}$$

Here Θ denotes the learnable parameters of the network. These forecasted trajectories are subsequently integrated into the delay, energy, and reliability models of the forecast-driven task offloading framework.

4.1.2. Multi-period feature extraction

TimesNet is adopted because it provides both strong forecasting performance and a modeling mechanism that matches MEC telemetry characteristics. On the one hand, TimesNet has been reported to achieve consistent state-of-the-art performance across mainstream time series tasks, including short and long-term forecasting (Wu et al. 2023). On the other hand, MEC telemetry (e.g., server workload and failure-related signals) often exhibits overlapped periodicities together with nonstationary variations. In each TimesBlock, dominant periods are identified in the frequency domain and the 1D sequence is reorganized into period-aligned 2D views to expose intra-period and inter-period dynamics, followed by a multi-scale convolutional unit to extract multi-scale temporal patterns. In our datasets, the forecast module shows stable convergence and a monotonic SMAPE decrease that reaches a plateau (Fig. 4), providing reliable short-horizon predictive cues for the downstream offloading policy. For clarity in the main text, we focus on the input–output interface and the integration into the offloading context, while reporting the detailed backbone formulation in Appendix A for completeness and reproducibility.

4.1.3. Spectral attention fusion

TimesNet further fuses multi-period representations using an amplitude-guided soft weighting that emphasizes the most informative periodic components under the current operating regime (details in Appendix A).

4.1.4. F orecast head

After stacking multiple TimesBlocks, a lightweight decoder outputs the H-step forecast $\hat{\mathbf{Y}}_t^{(m)} \in \mathbb{R}^{H\times C_{\text{pred}}}$ of each server's workload and failure rate, details are deferred to Appendix A.

4.1.5. Reliability integration

To integrate the forecasted failure dynamics into the reliability model, the failure rate trajectory is aggregated into a single effective failure rate indicator:

$$\hat{\rho}_t^{(m)} = \frac{1}{H}\sum_{\tau=1}^{H} \hat{\rho}_{t+\tau}^{(m)}. \tag{29}$$

This aggregated reliability metric is directly injected into the execution reliability model for tasks assigned to ES_m, while the forecasted workload sequence is incorporated into the delay and energy models. When task D_i is offloaded to ES_m, its execution reliability is computed as:

$$\mathcal{R}_i^{\text{exe}} = \exp\left(-\theta\, \hat{\rho}_t^{(m)}\, C_i D_i\right). \tag{30}$$

The forecasted load $\hat{L}_m(t)$ is similarly incorporated into the delay and energy models. Through these integrations, the system evaluates each offloading action using anticipated server conditions, enabling reliability-aware and proactive task scheduling.

4.1.6. Training objective

The TimesNet forecast module is trained offline using historical multivariate sequences collected from all ESs. Given a training set containing R input windows and their corresponding forecast horizons, the model parameters Θ are optimized by minimizing the mean squared error between ground truth future trajectories and forecasted outputs:

$$\mathcal{L}_{\text{pred}}(\Theta) = \frac{1}{RHMC_{\text{pred}}}\sum_{r=1}^{R}\sum_{m=1}^{M}\sum_{\tau=1}^{H}\left\|\hat{\mathbf{y}}_{t_r+\tau}^{(r,m)} - \mathbf{y}_{t_r+\tau}^{(r,m)}\right\|_2^2. \tag{31}$$

After training, the module operates online to generate $\hat{\mathbf{Y}}_t^{(m)}$ for each server, providing forecasted context for the forecast-driven task offloading policy.

In summary, the TimesNet module provides short-horizon multi-step forecasts of server load and failure dynamics, which are directly injected into our delay/energy/reliability evaluation to close the forecast-to-decision loop for proactive and reliability-aware offloading.

4.2. Forecast-driven online task offloading with linucb

To enable forecast-driven online offloading, we formulate the decision process as a contextual multi-armed bandit problem and adopt the Linear Upper Confidence Bound (LinUCB) algorithm. The action space is denoted as $\mathcal{A}$, containing local execution, the N ESs, and the cloud endpoint, giving $|\mathcal{A}| = N + 2$. LinUCB is particularly suitable for this setting because, compared with existing online offloading methods that rely on deep reinforcement learning or complex forecasting controllers, it offers a lightweight and sample efficient alternative. Its linear structure enables fast adaptation to nonstationary environments, and the exploration based on confidence naturally handles uncertainty in the forecasted load and reliability. These properties are particularly valuable in our forecast-driven setting, where queue lengths, wireless links, and server conditions fluctuate rapidly. Recent system-level studies in hierarchical and multi hop MEC further confirm that contextual bandit learners consistently improve delay and energy performance under dynamic operating conditions, which supports using LinUCB as an effective and practical decision engine within our forecast-driven task offloading framework.

4.2.1. LinUCB mathematical model

In LinUCB, the offloading decision at each time slot t is guided by a linear reward model that evaluates every candidate action $a \in \mathcal{A}$ using a context vector $x_{t,a} \in \mathbb{R}^d$ constructed from system features and forecast-driven indicators. The expected reward is modeled as a linear function:

$$\hat{r}_{t,a} = \theta^{\top} x_{t,a}. \tag{32}$$

Here θ is an unknown parameter vector to be learned online. LinUCB maintains a regularized covariance matrix and a cumulative reward vector, and computes the parameter estimate:

$$A_t = \lambda I + \sum_{s=1}^{t-1} x_{s,a_s} x_{s,a_s}^{\top}, \qquad b_t = \sum_{s=1}^{t-1} x_{s,a_s} r_s, \tag{33}$$

$$\theta_t = A_t^{-1} b_t. \tag{34}$$

For each action a, LinUCB evaluates an upper confidence bound:

$$UCB_{t,a} = \theta_t^{\top} x_{t,a} + \alpha\sqrt{x_{t,a}^{\top} A_t^{-1} x_{t,a}}. \tag{35}$$

Here α controls the balance between exploration and exploitation. The selected action is:

$$a_t = \arg\max_{a\in\mathcal{A}} UCB_{t,a}. \tag{36}$$

After observing the execution feedback, the algorithm updates A_t and b_t and continues to refine θ_t. This online update mechanism allows LinUCB to rapidly adapt to evolving network and server conditions while maintaining interpretable decision boundaries.

4.2.2. Reward design

The reward signal guides LinUCB toward selecting offloading actions that minimize latency and energy consumption while maintaining high execution reliability. Following the system model, the instantaneous performance of a task D_i executed under action a at slot t is characterized by its total delay T_i^{total}, energy cost E_i^{total}, and reliability $\mathcal{R}_i^{total}$. To align the learning objective of LinUCB with the global optimization goal, we convert these metrics into a scalar reward by defining:

$$r_{t,a} = -\left(\alpha\, T_i^{total} + \beta\, E_i^{total} - \gamma\, \mathcal{R}_i^{total}\right), \tag{37}$$

Here α, β, and γ are positive weighting coefficients that reflect the relative importance of delay, energy, and reliability. This formulation encourages the learner to prefer actions that reduce latency and energy consumption while improving reliability. After executing action a_t

and obtaining the corresponding performance metrics, the computed reward r_{t,a_t} is fed back to LinUCB to update its model parameters.

The reward design provides two advantages in the forecast-driven setting. First, it directly embeds the effect of forecasted load and failure conditions into the decision feedback through T_i^{total} and $\mathcal{R}_i^{total}$. Second, the linear combination aligns naturally with LinUCB's linear reward model, ensuring numerical stability and allowing the learner to quickly capture the tradeoffs among delay, energy, and reliability under time-varying edge environments.

4.2.3. Context construction

The effectiveness of LinUCB depends critically on the construction of an expressive and numerically stable context vector that captures the dominant factors influencing delay, energy, and reliability. For each action $a \in \mathcal{A}$ at time slot t, we construct a block structured context vector $x_{t,a}$ that integrates task attributes, forecasted server states, and system-level features. The full context vector is defined as:

$$x_{t,a} = \begin{bmatrix} z_t \\ u_t \\ b_d(d_t) \\ b_h(t) \\ e_a \\ 1 \end{bmatrix}. \tag{38}$$

Here the block z_t contains normalized core features,

$$z_t = \Lambda^{-1} \begin{bmatrix} D \\ R \\ f_r \\ C \\ q_n \end{bmatrix}, \qquad \Lambda = \mathrm{diag}\big(\kappa_D,\ \widehat{R}_{\max},\ \kappa_{fr},\ \kappa_C,\ 1\big), \tag{39}$$

with D representing task data volume, R the link rate, f_r a failure risk prior, C the computation intensity, and q_n a normalized queue measure. The diagonal matrix Λ ensures numerical conditioning for the linear estimator. To capture key interactions between computation and communication, the interaction block is defined as:

$$u_t = \begin{bmatrix} (D \cdot C)/\kappa_{DC} \\ (R/\widehat{R}_{\max}) \cdot q_n \end{bmatrix}. \tag{40}$$

This explicitly models the joint influence of input volume and compute demand, as well as the coupling between channel quality and queue pressure. Distance dependent transmission behavior is encoded using a three bin one hot vector:

$$b_d(d_t) = \begin{bmatrix} \mathbb{I}(d_t < \tau_{d,1}) \\ \mathbb{I}(\tau_{d,1} \le d_t < \tau_{d,2}) \\ \mathbb{I}(d_t \ge \tau_{d,2}) \end{bmatrix}, \tag{41}$$

Here $\tau_{d,1}$ and $\tau_{d,2}$ separate close, medium, and far communication ranges. Temporal variation is represented by a four dimensional time of day encoding $b_h(t) \in \{0,1\}^4$. The vector e_a is an action identifier that marks ES arms using one hot encoding, with local and cloud actions mapped to the zero vector to avoid artificial similarity between unrelated arms. The final constant element aligns the design with standard linear models that include a bias term.

4.2.4. Online decision and update mechanism

Based on the forecast-driven contexts and the linear reward model described above, the LinUCB module performs online action selection and parameter updates in each time slot. For every candidate action, the algorithm evaluates an upper confidence score that balances exploitation of the estimated reward and exploration driven by model uncertainty. The action with the highest score is executed, and the resulting composite cost is mapped into a normalized reward. The model parameters of the selected arm are then updated using a ridge regularized incremental rule, allowing the learner to quickly adapt to evolving load, link, and reliability conditions without retraining.

The complete procedure is summarized in Algorithm 1. This algorithm operates in an online manner and iteratively determines the execution target for each arriving task. At every time slot, the system first obtains short-horizon workload and failure rate forecasts from the TimesNet module and combines them with instantaneous measurements to construct a context vector for each candidate action. Using these contexts, the algorithm computes a linear reward estimate and an uncertainty term from the ridge regularized covariance maintained for each arm. The upper confidence score is obtained by combining these two components, and the action with the highest score is selected for execution. After the task is completed, the observed composite cost is converted into a clipped reward, and the ridge statistics associated with the chosen arm are updated to refine its reward estimate.

The overall procedure is lightweight and numerically stable, which makes it suitable for edge deployment. The per step computational cost scales with $\mathcal{O}(|\mathcal{A}|d^2)$, where d is the dimension of the forecast-driven context vector. This complexity arises from evaluating the UCB score and performing ridge style incremental updates. Such efficiency aligns with existing studies showing that contextual bandit algorithms can satisfy real-time constraints while improving delay and energy tradeoffs under nonstationary MEC conditions. Consequently, the proposed forecast-driven LinUCB module maintains high responsiveness in dynamic environments while remaining compatible with the resource limitations of edge systems.

In our default MEC setting (Table 2), the action set includes local execution, M candidate edge servers, and the cloud, hence $|\mathcal{A}| = M + 2 = 7$. The per-slot time complexity scales as $\mathcal{O}(|\mathcal{A}|d^2)$, and the memory overhead is $\mathcal{O}(|\mathcal{A}|d^2)$ since each arm maintains one $d \times d$ ridge covariance matrix and one d-dimensional vector. Moreover, the per-slot update only touches the selected arm and can be implemented with an incremental inverse update (e.g., via Sherman–Morrison), keeping the update overhead at $\mathcal{O}(d^2)$.

5. Experimental evaluation

5.1. Experimental setup

All experiments are conducted within a unified simulation framework that emulates a three-tier MEC environment consisting of UDs, idle execution nodes, and multiple ESs. The system model, constraints, and evaluation criteria remain consistent across all compared methods to ensure fairness. All experiments are executed on a workstation equipped with one NVIDIA GeForce RTX 4090 GPU with 24 GB memory, and all learning modules, including TimesNet and LinUCB components, are implemented using PyTorch 2.0.1.

5.1.1. Baselines and variable design

We compare the proposed forecast-driven LinUCB approach with several representative online decision-making baselines, including the softmax bandit algorithm (Wu et al., 2025), UCB (Zhong et al., 2025) and LinUCB (Shustova et al., 2025). All methods operate under the same execution budget and system constraints. The experiments vary three primary dimensions: load intensity, device population, and transmission capacity. The per device task arrival rate is gradually increased to examine scalability under heavy load, while the number of devices is varied to evaluate performance under increasing competition for edge resources. During each experiment, we track the overall system cost and the number of iterations required for convergence, enabling comparison of stability and efficiency across baselines.

5.1.2. Forecast model configuration

The forecast component adopts TimesNet as the backbone. All hyperparameters follow the default implementation unless otherwise specified. The input window length is set to 96, the decoder warm up

Algorithm 1 Forecast-driven Online Task Offloading via LinUCB

Require: action set $\mathcal{A} = \{\text{local}\} \cup \{\text{edge}_i\}_{i=1}^{N} \cup \{\text{cloud}\}$, ridge strengths λ_0, λ_r, reward scale κ_r, exploration schedule $\alpha(t)$, forecast module TimesNet$(\cdot)$

Ensure: chosen actions $\{a_t^\star\}_{t=1}^{T}$, and arm parameters $\{A[a], b[a], \hat{\theta}[a]\}_{a\in\mathcal{A}}$

for all $a \in \mathcal{A}$ **do**
  $A[a] \leftarrow \lambda_0 I_d$ ▷ ridge initialized covariance
  $b[a] \leftarrow \mathbf{0}_d$ ▷ first order reward accumulator
end for
for $t \leftarrow 1$ **to** T **do**
  obtain instantaneous telemetry and task meta: (D, R, f_r, C, q_n, d_t)
  obtain short-horizon forecasts for each ES via

$$\{\hat{L}_m(t), \hat{\rho}_m(t)\}_{m=1}^{M} \leftarrow \text{TimesNet}\left(\mathbf{X}_t^{(m)}\right)$$

  for all $a \in \mathcal{A}$ **do**
    construct context vector

$$x[t,a] \leftarrow \textsc{BuildContext}(D, R, f_r, C, q_n, d_t, \{\hat{L}_m, \hat{\rho}_m\}, a)$$

    ▷ includes normalized core block, interaction block, bins, action id
    $\hat{\theta}[a] \leftarrow A[a]^{-1} b[a]$ ▷ current linear estimate
    $s[a] \leftarrow \sqrt{x[t,a]^\top A[a]^{-1} x[t,a]}$ ▷ uncertainty term
    $\text{score}[a] \leftarrow x[t,a]^\top \hat{\theta}[a] + \alpha(t)\, s[a]$
  end for
  $a_t^\star \leftarrow \arg\max_{a\in\mathcal{A}} \text{score}[a]$ ▷ select best arm
  execute $a_t^\star$ and observe composite cost $C_{\text{obs}} \triangleq C_t(a_t^\star)$
  compute clipped reward

$$r \leftarrow \text{clip}\left(1 - \frac{C_{\text{obs}}}{\kappa_r},\ -1,\ 1\right)$$

  Update selected arm statistics:
  $A[a_t^\star] \leftarrow A[a_t^\star] + x[t,a_t^\star]\, x[t,a_t^\star]^\top + \lambda_r I_d$
  $b[a_t^\star] \leftarrow b[a_t^\star] + r\, x[t,a_t^\star]$
  optionally update cached inverse $A[a_t^\star]^{-1}$ using Sherman Morrison for efficiency
end for
return $\{a_t^\star\}_{t=1}^{T}$, $\{\hat{\theta}[a]\}_{a\in\mathcal{A}}$

length to 48, and the forecast horizon to 96. The number of principal periodic branches is 5, and the multi-scale convolution block contains 6 parallel kernels. The embedding dimension is 32 and the intermediate dimension is 32 so that the forecasting module remains lightweight in the integrated framework. Two TimesBlocks are stacked. Time feature encoding follows the `timeF` mode with hourly frequency encoding, and dropout is set to 0.1. The model is trained using Adam with a learning rate of 1×10^{-3} and batch size of 16. Mixed precision is disabled and all runs fix random seeds.

In the edge computing experiments(Wang et al., 2025), the forecasts produced by TimesNet are consumed by a contextual bandit offloading policy that operates in a simulated MEC environment with ten UDs, five ESs, and a remote cloud server. The UD CPU frequency is set to 0.8 GHz, ES CPU frequencies are linearly spaced between 8 and 12 GHz, and the cloud CPU frequency is 40 GHz. The uplink channel bandwidth is configured as 30 MHz with transmit power 0.25 W and noise power spectral density 10^{-9} W/Hz.

To avoid ambiguity, we clarify the rationale of the power-related parameters in Table 2. The transmit power (0.25 W) represents the nominal uplink power budget of a mobile device, which sets the operating scale of the wireless energy term in the cost model (Nie et al., 2025b). The noise power spectral density (10^{-9} W/Hz) is treated as an effective noise level that absorbs background noise and interference, so that the resulting SNR range remains within a realistic operating regime for MEC offloading. Wu et al. (2022) Moreover, the computation-energy coefficients (k_l, k_e, k_c) follow the standard CMOS dynamic power model ($E = \kappa f^2 \times$ cycles) and are chosen to match commonly used magnitudes in MEC simulation studies, ensuring that the energy term remains physically interpretable when combined with latency and reliability. Zhou et al. (2025)

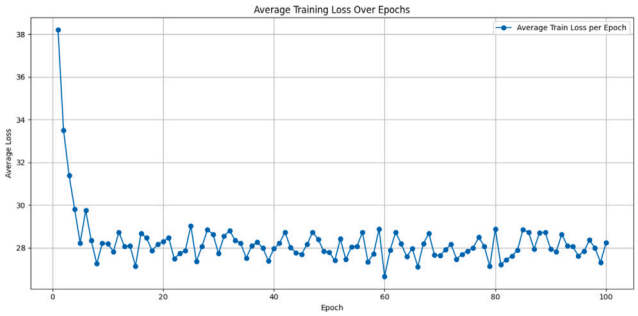
Table 2
Configuration of the TimesNet-based forecasting module and the MEC system. Power-related parameters (transmit power, noise PSD, and energy coefficients) are set to typical MEC operating magnitudes and are used to define the scale of wireless/computation energy terms in the system-level objective (Nie et al., 2025b; Wu et al., 2022; Zhou et al., 2025).

Parameter	Value
TimesNet-based forecast module configuration	
Input window length (L)	96
Decoder warm up length	48
Forecast horizon (H)	96
Number of periodic branches	5
Number of convolution branches	6
Embedding dimension	32
Intermediate channel dimension	32
Number of TimesBlocks	2
Time feature encoding	`timeF`
Dropout rate	0.1
Optimizer	Adam
Learning rate	1×10^{-3}
Batch size	16
Training epochs (max)	100
MEC system configuration	
Number of UDs	10
Number of ESs (M)	5
Number of LinUCB arms	$M+2$
UD CPU frequency	0.8 GHz
ES CPU frequency	$8\sim12$ GHz
Cloud CPU frequency	40 GHz
UD energy coefficient	1×10^{-28}
ES energy coefficient	1×10^{-29}
Cloud energy coefficient	1×10^{-29}
System bandwidth	30 MHz
Transmit power	0.25 W
Noise power spectral density	1×10^{-9} W/Hz
Reliability parameter	0.5
Bit error rate	5×10^{-7}
Baseline failure intensity	6.364×10^{-2}

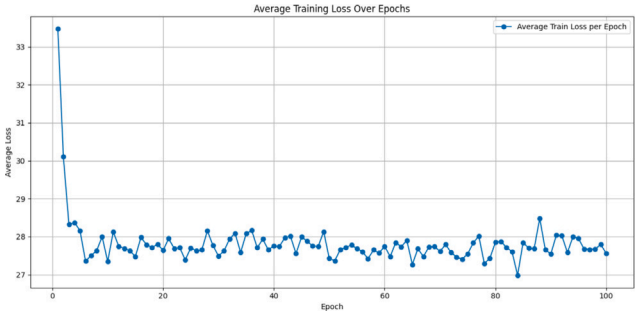
The reliability model adopts $\theta = 0.5$ with bit error rate 5×10^{-7}, and Table 2 reports a baseline failure intensity 6.364×10^{-2} as a nominal scale for simulator configuration. Importantly, the decision-time failure-rate input is forecasted by the TimesNet module from real telemetry (SMD included): at each slot, TimesNet outputs the short-horizon failure-rate forecast $\hat{F}_m(t)$ (together with $\hat{L}_m(t)$), which is injected into the reliability evaluation (Eq. (13)) used by the downstream offloading policy. For controlled and reproducible comparisons, we use a homogeneous risk profile across ESs in the simulator (i.e., the same forecasted failure trajectory is shared across servers at a given time), so that performance differences primarily reflect the forecast-to-decision mechanism rather than risk heterogeneity.

5.1.3. Forecast datasets

We evaluate forecasted performance using two real multivariate time series datasets covering edge workload behavior and server operational dynamics. The first dataset (ECW) (Huang et al., 2023) consists of logs collected from a commercial edge cloud platform between August and October 2022, released as monthly tensors. ECW-08 has size $797\times720\times16$ and ECW-09 has size $1022\times720\times16$. Each record includes 16 features, comprising both dynamic variables (e.g., upload bandwidth, time index) and static attributes (e.g., core count, memory configuration). All features are normalized to the [0,1] range for training and restored for system-level evaluation. The dataset exhibits strong intraday and weekly periodicities together with abrupt changes



(a) SMAPE Obtained from the ECW Dataset Input into TimesNet



(b) SMAPE Obtained from the SMD dataset Input Into TimesNet

Fig. 4. Performance of Different Datasets in TimesNet.

caused by application switching, making it suitable for evaluating forecast-driven scheduling.

The second dataset (SMD) is a five weeks multivariate anomaly detection benchmark that includes 28 servers and 38 monitoring channels (Su et al., 2019). It provides training/testing splits and ground truth anomaly labels, enabling evaluation under noisy or disturbed system conditions and assessing robustness to irregular dynamics. In our framework, the failure-rate signal is derived from telemetry-driven failure/anomaly evidence. Specifically, we use the SMD anomaly labels as supervision to construct a failure-event indicator and its corresponding failure-intensity series, which serves as the forecasting target for the failure-dynamics channel. TimesNet is then trained to output short-horizon multi-step forecasts of failure dynamics, yielding the per-slot predicted failure rate $\hat{F}_m(t)$ that is used by the offloading algorithm.

5.1.4. Evaluation metrics

Forecasted performance is measured using the Symmetric Mean Absolute Percentage Error (SMAPE). On the system side, overall cost (latency, energy, and reliability) is used as the primary metric. Additional variables such as device count, upload delay, transmission energy, and reliability variation are analyzed to study their effects on system performance.

5.2. Experimental results and analysis

5.2.1. Forecast model experiment

We first evaluate the forecast component that drives the proposed offloading framework. Under a unified data preprocessing pipeline and fixed hyperparameters, TimesNet is trained twice independently and evaluated once for validation. As shown in Fig. 4(a), the first training run exhibits a rapid loss decrease from approximately 38.2 to 28.3 within the first ten epochs, followed by convergence within a narrow band from 27.5 to 28.5. The second run in Fig. 4(b) demonstrates similar behavior, where the loss decreases from around 33.5 and stabilizes between 27.2 and 27.9 after twenty epochs. The consistency between the two runs confirms that the chosen window size and learning parameters form a reliable convergence region suitable for system-level experimentation.

Meanwhile, SMAPE decreases monotonically and reaches a stable plateau in later epochs. These results indicate that TimesNet successfully captures the dominant periodic and local variation patterns in multivariate workload traces. The stable forecast accuracy provides a dependable basis for actionable short-horizon load and reliability estimates that are used by the offloading algorithm.

5.2.2. Convergence performance analysis

To verify the effectiveness and stability of the proposed forecast-driven offloading strategy, we analyze the convergence behavior of LinUCB under the unified system and forecast setup. Fig. 5 reports the cumulative average overall cost after discarding the first twenty cold start iterations. The curve decreases rapidly from roughly 390 to 300, exhibits a short transient fluctuation near the one hundredth round, and continues to decline steadily thereafter. Around the six hundredth round, the cost enters a narrow band between 278 and 282, indicating the onset of stable behavior. After approximately fifteen hundred rounds, the curve stabilizes between 275 and 278.

This trajectory is consistent with the underlying LinUCB mechanism. As the number of context and reward interactions increases, parameter estimation becomes more accurate and the confidence intervals contract. The exploration coefficient gradually decays, allowing the algorithm to transition smoothly into exploitation. Combined with the stable short term forecasts produced by TimesNet, LinUCB rapidly absorbs contextual information such as forecasted load, link quality, queue intensity, and expected reliability. This leads to efficient convergence and long-term cost stability, demonstrating that LinUCB is a viable and robust offloading strategy in dynamic MEC environments.

5.2.3. Performance comparison with baseline algorithms

After establishing the convergence and stability of the proposed forecast-driven offloading scheme, we compare its performance with representative online baselines including Softmax, UCB1, and standard

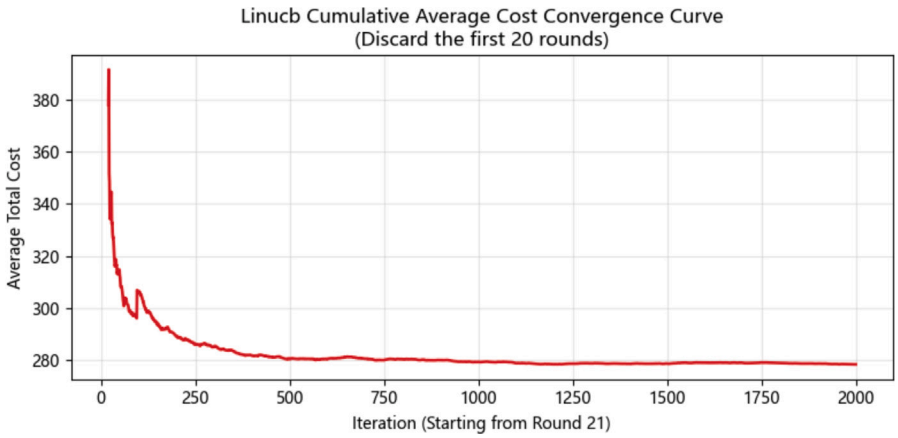
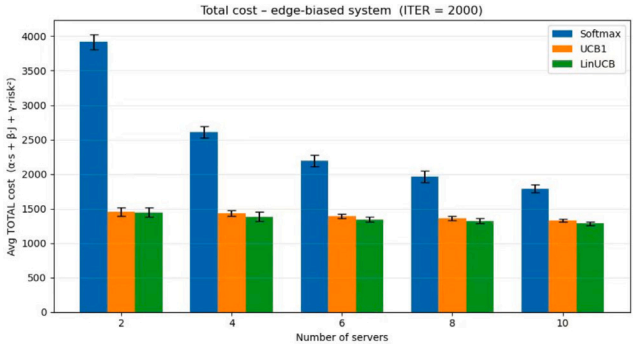
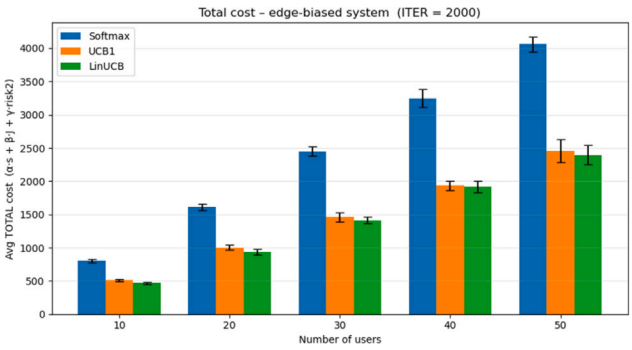


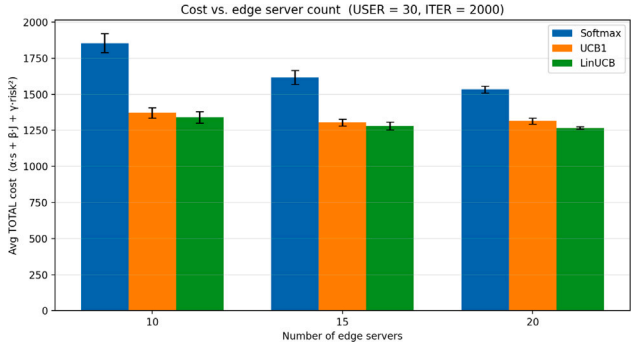
Fig. 5. Convergence Performance analysis of a LinUCB-based Algorithm.



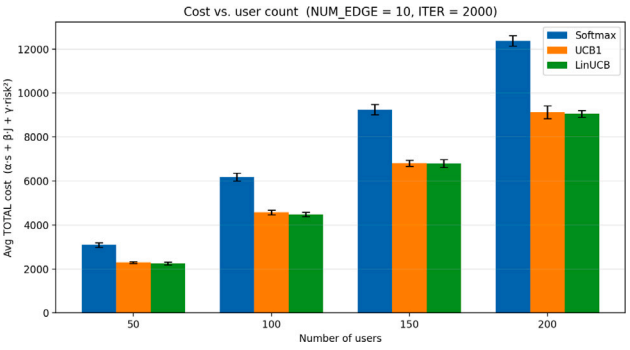
(a) Impact of Server Count on System Cost



(b) Impact of UD Count on System Cost



(c) Extended Server Count Scaling ($N \in \{10, 15, 20\}$)



(d) Extended UD Scaling ($U \in \{50, 100, 150, 200\}$, $N = 10$)

Fig. 6. System Cost Comparison Under Different Server and UD Scales (including extended ranges).

LinUCB, as shown in Fig. 6. Two sets of scalability experiments are conducted by varying the number of available ESs and the number of UDs, respectively.

To further validate scalability beyond the original ranges, we additionally extend the server-count setting to $N \in \{10, 15, 20\}$ with USER= 30 (Fig. 6(c)) and extend the user-count setting to $U \in \{50, 100, 150, 200\}$ under a fixed $N = 10$ (Fig. 6(d)), while keeping the same simulation protocol (ITER= 2000 and 30 random seeds).

From an algorithmic perspective, the online selection and update in our framework remain lightweight. In particular, the per-slot complexity of the LinUCB decision layer scales as $\mathcal{O}(|\mathcal{A}|d^2)$, where $|\mathcal{A}| = N + 2$ corresponds to local execution, N candidate edge servers, and the cloud endpoint, and d is the context dimension. This implies a predictable polynomial increase of online overhead when the candidate server set expands. Furthermore, for more complex MEC topologies such as hierarchical or multi-hop deployments, the same forecast-driven decision principle can be applied by organizing candidate nodes into layer-wise action sets and performing layer-wise selection with feedback updates, which we leave as a practical extension direction.

Server count scaling: Increasing the number of available ESs from two to ten enhances the aggregate computation capacity and expands the scheduling flexibility of the system. As shown in Fig. 6(a), the overall system cost decreases monotonically for all evaluated methods as the server count grows, although the marginal benefit gradually diminishes. In the range of two to ten servers, Softmax reduces cost from around 3900 to approximately 1800, UCB1 reduces cost from about 1450 to 1320, and LinUCB reduces cost from roughly 1450 to 1250. Although Softmax exhibits the largest absolute drop, its initial cost is significantly higher and its variance is more pronounced, especially in the low server regime.

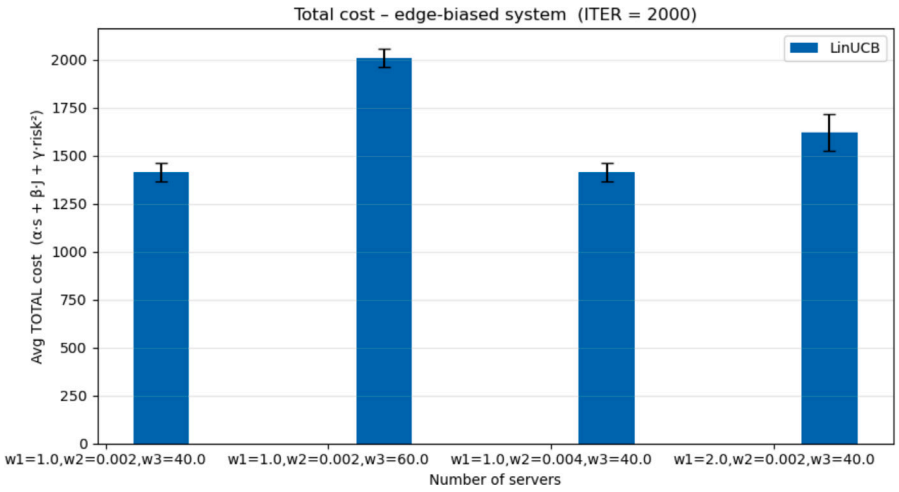


Fig. 7. Weight sensitivity and interpretation.

Table 3
Feature ablation of the LinUCB context (leave-one-block-out). Results are reported as mean ± 95% CI over $N = 5$ random seeds. The context vector is unit-norm normalized before LinUCB scoring and update to avoid scale-induced exploration bias. Δ denotes the relative change in average system cost compared with the full-feature setting.

Variant	Avg. system cost ↓	Δ vs. Full
Full	1394.11 ± 66.47	+0.00%
w/o core block	1350.74 ± 32.30	−3.11%
w/o forecast block	1388.03 ± 72.61	−0.44%
w/o interaction block	1375.72 ± 68.81	−1.32%
w/o **distance block**	**1437.14 ± 93.06**	**+3.09%**
w/o **time block**	**1436.64 ± 93.57**	**+3.05%**
w/o arm-id block	1395.10 ± 65.62	+0.07%

LinUCB consistently establishes the lowest cost envelope across all server configurations and maintains shorter error bars, indicating strong stability and robustness to variations in server availability and resource distribution. When the server count exceeds eight, the improvement for all methods diminishes substantially, signaling entry into a resource redundancy region in which expanding server capacity provides limited marginal gains. Nevertheless, LinUCB remains the most stable and efficient method throughout this regime.

UD scaling: We then evaluate robustness under increasing UD load by scaling the number of UDs from ten to fifty. As shown in Fig. 6(b), system cost increases for all algorithms as user demand grows. At ten devices, Softmax and LinUCB achieve costs near 1000, while UCB1 is around 1300. At fifty devices, Softmax and UCB1 increase dramatically to approximately 3100 and 3150, whereas LinUCB maintains a substantially lower cost near 2600.

A comparison of the slope of each curve reveals that LinUCB experiences the slowest growth in cost as UD load intensifies. This indicates that LinUCB can more effectively balance delay, energy consumption, and reliability penalties under high load. UCB1 displays significantly longer error bars between thirty and fifty devices, suggesting higher sensitivity to nonstationary system dynamics and fluctuating link conditions. Softmax demonstrates both higher variance and a faster increase in mean cost, highlighting weaker adaptability as user demand scales.

Combined with the convergence performance analysis in Section 5.2.2, these results confirm that the proposed forecast-driven LinUCB strategy provides superior scalability, robustness, and cost efficiency under both server side and user side load variations. Its ability to leverage forecasted load and reliability while maintaining stable long-term performance underscores its practical viability in real-world MEC deployments.

We further analyze the contribution of each feature block in the LinUCB context through a leave-one-block-out ablation study. One block is removed (set to zero) at a time while keeping all other experimental settings identical. To ensure a fair comparison across ablation variants, we apply unit-norm normalization to the context vector before LinUCB scoring and update. All results are averaged over $S = 5$ independent runs with different random seeds, and We report the mean and the 95% confidence interval (CI) over $S = 5$ seeds. The results are summarized in Table 3.

Note that the leave-one-block-out ablation is reported with mean ± 95% CI over $S = 5$ seeds due to its higher computational budget, whereas the scalability experiments in Fig. 6 and the component-wise breakdown are summarized over 30 seeds using median ± robust std (IQR/1.35) for robustness.

In Fig. 6, the system cost is computed by the overall objective in Eq. (15a). To make the trade-off transparent, we additionally report the component-wise metrics under the same settings as Fig. 6, including per-task average latency, per-task average energy consumption, and per-task average reliability. The results are summarized in Tables 4 and 5 Each value is summarized as median ± robust std (IQR/1.35) across 30 random seeds, following the same simulation protocol (USER=30, ITER=2000).

5.2.4. Weight sensitivity and interpretation

Finally, we study the sensitivity of the overall cost to the objective weights associated with delay, energy, and reliability. Under a fixed iteration budget of $T = 2000$ in an edge-biased scenario, four weight combinations are evaluated as summarized in Fig. 7. The baseline setting $(\alpha, \beta, \gamma) = (1.0, 0.002, 40.0)$ produces an average total cost close to 1.4×10^3 with short error bars, indicating stable behavior across runs.

Increasing the reliability weight from $\gamma = 40.0$ to $\gamma = 60.0$ raises the cost to nearly 2.0×10^3, revealing that the offloading policy is most sensitive to the reliability component. Higher reliability emphasis encourages safer execution paths but cannot completely mitigate the

Table 4
Component-wise comparison (Latency and Energy) under representative server scales. Values are per-task averages. Each entry is median ± robust std (IQR/1.35) across 30 seeds.

N	Avg latency			Avg energy (J)		
	Softmax	UCB1	LinUCB	Softmax	UCB1	LinUCB
2	11.211 ± 1.901	12.176 ± 3.297	13.872 ± 3.864	0.708 ± 0.107	0.728 ± 0.142	0.735 ± 0.157
6	7.949 ± 1.912	7.624 ± 2.258	7.879 ± 1.928	0.696 ± 0.126	0.726 ± 0.168	0.732 ± 0.196
10	4.892 ± 0.843	4.591 ± 0.518	4.662 ± 1.309	0.732 ± 0.125	0.726 ± 0.114	0.716 ± 0.176

Table 5
Component-wise comparison (Reliability) under representative server scales. Values are per-task averages. Each entry is median ± robust std (IQR/1.35) across 30 seeds.

N	Softmax	UCB1	LinUCB
2	0.742796 ± 0.000124	0.741212 ± 0.000120	0.741223 ± 0.000111
6	0.741799 ± 0.000136	0.741293 ± 0.000075	0.741291 ± 0.000064
10	0.741669 ± 0.000145	0.741341 ± 0.000137	0.741313 ± 0.000075

Table 6
Runtime overhead of the online LinUCB decision layer (decision computation time, not system execution latency). We report the per-task-step computational time of UCB scoring and selected-arm update in milliseconds under the extended-scale configuration (N = 10, USER = 30). Statistics are aggregated over 30 seeds after discarding the first 20 rounds.

Module	Mean (ms)	Median (ms)	P95 (ms)
UCB scoring (all arms)	0.218643	0.217478	0.229384
Selected-arm update	0.006540	0.006502	0.006879
Total (scoring+update)	0.225183	0.223971	0.236247

associated delay and energy penalties. Adjusting the energy weight from $\beta = 0.002$ to $\beta = 0.004$ yields negligible change, reflecting the limited influence of energy consumption under the current load and link configuration. Doubling the delay weight from $\alpha = 1.0$ to $\alpha = 2.0$ increases the cost by approximately fifteen percent, indicating a stronger aversion to high queue or low rate decisions that elevates the total cost.

These findings provide two practical tuning guidelines. First, the reliability weight requires careful calibration since it dominates the cost function and influences policy behavior most strongly. A narrow range around forty to sixty offers a balanced tradeoff aligned with service level requirements. Second, the energy weight can be fixed at a small constant because its contribution remains small across a wide range of settings. The remaining experiments therefore adopt fine adjustments to γ supplemented by minor refinements to α, while β is held constant to maintain a stable balance between delay and reliability.

5.2.5. Forecast error sensitivity analysis

All system-level results are obtained under the simulated MEC environment described in Section 5.1. To quantify how load prediction errors affect the offloading performance in the forecast-to-decision loop, we conduct a controlled sensitivity test by injecting multiplicative biases into the load forecast used by the decision module.

Specifically, we scale the forecasted load as $\hat{L}_m(t) \leftarrow (1+\delta)\hat{L}_m(t)$ with $\delta \in \{-0.6, -0.3, 0, +0.3, +0.6\}$, while keeping the task trace and system parameters unchanged. Fig. 8 reports the average total cost under different bias levels. The cost increases monotonically as the injected bias shifts from negative to positive. This trend is consistent with Eqs. (7) and (8), where a larger $\hat{L}_m(t)$ reduces the effective service rate and increases the edge computation latency. It is also consistent with Eq. (15d), where the remaining edge capacity decreases with $\hat{L}_m(t)$, which can further restrict feasible edge execution and increase the overall system cost. These results indicate that the proposed offloading strategy is sensitive to load forecast errors, and the impact becomes more pronounced under larger bias magnitudes.

5.2.6. Computational overhead and response latency

To empirically validate the lightweight and real-time property, we measure the runtime overhead of the online LinUCB decision layer under the same configuration as the extended-scale setting (NUM_EDGE = N = 10, USER = 30, ITER = 2000). We instrument the per-task decision pipeline and separately report the latency of (i) UCB scoring over all arms and (ii) the selected-arm update. All results are aggregated over 30 random seeds after discarding the first 20 cold-start rounds, and we report mean/median/P95 in milliseconds in Table 6.

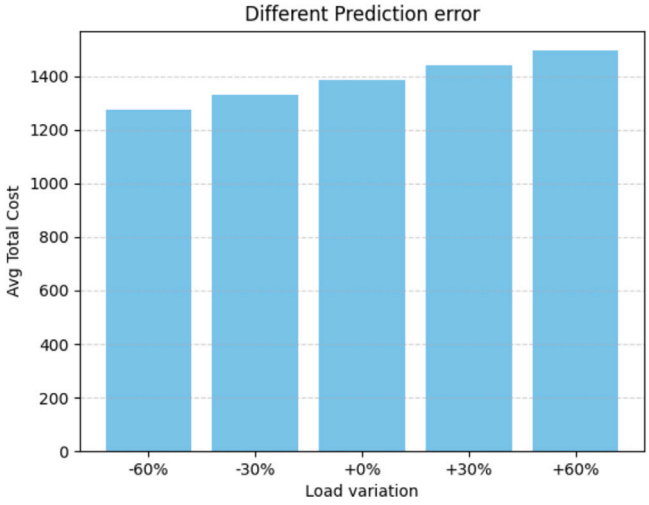


Fig. 8. Simulation-based sensitivity of the average total cost to controlled perturbations of the forecasted load, with $\delta \in \{-0.6, -0.3, 0, +0.3, +0.6\}$.

6. Conclusion

This paper presented a forecast-driven task offloading framework that integrates multi-period time-series forecast with lightweight online decision-making for reliable and adaptive MEC. A TimesNet-based forecast module provides robust short-horizon forecasts of server load and failure dynamics, and a LinUCB-based offloading policy leverages these forecasted contexts to enable low complexity and rapidly adaptive offloading. Experiments on real-world workload traces and simulated

failure scenarios show that the proposed method reduces task failures, lowers system cost, and achieves faster convergence than state-of-the-art optimization and learning-based baselines, demonstrating the effectiveness of coupling forecast with online learning for proactive MEC management.

Building on the proposed forecast-to-decision closed loop, which couples a TimesNet-based forecasting module with a lightweight LinUCB-based online decision layer, we identify several natural extensions. On the forecasting side, incorporating spatial correlations across distributed edge nodes generalizes the current per-node prediction to spatiotemporal forecasting, providing richer predictive context for offloading. On the decision side, developing multi-agent contextual learning extends the current single-agent LinUCB policy to coordinated offloading, and integrating long-horizon risk-aware forecasting further improves robustness to rare but high-impact workload and failure variations.

CRediT authorship contribution statement

Ting Li: Writing – review & editing, Validation, Supervision, Resources. **Yinan Mi:** Writing – original draft, Visualization, Software, Data curation. **Kai Yang:** Writing – review & editing, Resources, Project administration, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was supported by the Natural Science Foundation of Guangxi Zhuang Autonomous Region, China (Grant No. 2025GXNSFBA069393), National Natural Science Foundation of China (Grant No. 62502100), and Open Project Program of Guangxi Key Laboratory of Digital Infrastructure, China (Grant No. GXD-INBC2024 01).

Appendix A. TimesNet backbone details

For completeness and reproducibility, this appendix reports the detailed TimesNet backbone formulation that is omitted from the main text, including multi-period feature extraction, spectral-attention fusion, and the forecast head.

A.1. Multi-period feature extraction

The multi-period feature extractor in TimesNet consists of three components: Period Discovery based on the discrete Fourier transform (FFT), Period to 2D mapping, and Multi-scale Convolutional Extraction. Together, these components identify dominant temporal periods, restructure the sequence into a period aligned representation, and extract multi-scale temporal patterns essential for modeling workload and failure dynamics.

FFT-based period discovery: The first step in each TimesBlock is to transform the one dimensional temporal representation into the frequency domain in order to identify dominant periodic components. Given the hidden representation from the previous layer, denoted as $\mathbf{H}^{(\ell-1)} \in \mathbb{R}^{L\times d}$, TimesNet applies a discrete Fourier transform along the temporal dimension:

$$\mathbf{F} = \mathrm{FFT}\big(\mathbf{H}^{(\ell-1)}\big) \in \mathbb{C}^{L\times d}. \tag{42}$$

For each frequency index f, the average spectral amplitude across all channels is computed as:

$$a_f = \frac{1}{d}\sum_{c=1}^{d}\left|\mathbf{F}_{f,c}\right|. \tag{43}$$

The model then selects the top-K frequencies with the highest amplitudes, denoted as $f_k{}_{k=1}^{K}$, and converts them into approximate temporal periods:

$$p_k = \left\lceil \frac{L}{f_k} \right\rceil, \quad k = 1,\dots,K. \tag{44}$$

This spectral mechanism enables TimesNet to automatically discover both short range and long range periodicities, which commonly arise from diurnal usage cycles, queueing oscillations, and bursty traffic patterns in MEC systems.

Period to 2D mapping: Because the window length L is not necessarily a multiple of the identified period p_k, the temporal sequence must be extended before folding into two dimensional form. A zero padding operator produces a padded sequence of length $L_k = p_k f_k$:

$$\tilde{\mathbf{H}}_k^{(\ell-1)} = \mathrm{Pad}\big(\mathbf{H}^{(\ell-1)}, L_k\big) \in \mathbb{R}^{L_k\times d}. \tag{45}$$

The padded sequence is reshaped into a 2D tensor that explicitly organizes the temporal axis into period positions and period repetitions:

$$\mathbf{Z}_k^{(\ell)} = \mathrm{Reshape}_{p_k,f_k}\big(\tilde{\mathbf{H}}_k^{(\ell-1)}\big) \in \mathbb{R}^{p_k\times f_k\times d}. \tag{46}$$

Multi-scale convolutional extraction: To model both fine grained variations within a period and broader dependencies across periods, the reshaped tensor $\mathbf{Z}_k^{(\ell)}$ is processed by a multi scale convolutional block:

$$\tilde{\mathbf{Z}}_k^{(\ell)} = \mathrm{MSUnit}\big(\mathbf{Z}_k^{(\ell)}\big). \tag{47}$$

Here $\mathrm{MSUnit}(\cdot)$ contains multiple convolution branches with kernel sizes such as 1×1, 3×3, and 5×5, followed by activation and normalization layers. This inception style design enriches the receptive field and allows the model to jointly capture local temporal transitions within each period and long-term interactions across repeated periods. After convolution, the processed tensor is folded back into the one dimensional temporal form using the inverse reshape operator, and the padded elements are removed to maintain the original window length:

$$\tilde{\mathbf{H}}_k^{(\ell)} = \mathrm{Trunc}\Big(\mathrm{Reshape}_{L_k}^{-1}\big(\tilde{\mathbf{Z}}_k^{(\ell)}\big), L\Big) \in \mathbb{R}^{L\times d}. \tag{48}$$

Each $\tilde{\mathbf{H}}_k^{(\ell)}$ thus encodes the temporal dynamics associated with the kth discovered period and serves as one component of the final period aware representation computed in the subsequent spectral attention fusion stage.

A.2. Spectral attention fusion

To adaptively integrate the different temporal periods extracted in the previous step, TimesNet employs a spectral attention mechanism. Since the relevance of short term and long-term periodic patterns varies across server operating conditions, the model assigns learnable attention weights to the top K periods detected by the FFT-based the analysis. Given the amplitude a_{f_k} of each selected frequency f_k, the normalized spectral attention weight is computed using a softmax function:

$$w_k = \frac{\exp(a_{f_k})}{\sum_{j=1}^{K}\exp(a_{f_j})}, \quad k = 1,\dots,K. \tag{49}$$

These weights determine the relative importance of each period specific representation $\tilde{\mathbf{H}}_k^{(\ell)}$ for the current server state. The final output of the TimesBlock is obtained by combining the weighted period features with a residual connection:

$$\mathbf{H}^{(\ell)} = \mathbf{H}^{(\ell-1)} + \sum_{k=1}^{K} w_k\,\tilde{\mathbf{H}}_k^{(\ell)}. \tag{50}$$

A.3. Forecast head

After passing through multiple stacked TimesBlocks, the model obtains a refined temporal representation that captures both intra period and inter period dynamics. Let the final hidden state of the forecast backbone be denoted as $\mathbf{H}^{(L_{\text{blk}})}$. The forecast head transforms this representation into a multi-step forecast of future server conditions, including workload and failure rate. A lightweight decoder, such as a linear projection or temporal convolution, is applied to produce the forecast horizon:

$$\hat{\mathbf{Y}}_t^{(m)} = \phi\left(\mathbf{H}^{(L_{\text{blk}})}\right). \tag{51}$$

Here $\phi(\cdot)$ denotes the decoder mapping. The forecasted sequence has the form $\hat{\mathbf{Y}}_t^{(m)} \in \mathbb{R}^{H \times C_{\text{pred}}}$, which contains the forecasted workload and failure rate vectors across the next H time steps for server ES_m.

Data availability

Data will be made available on request.

References

Chhabra, G.S., Bansal, A., Kumar, R., 2025. Task offloading strategy based on performance prediction in vehicular edge computing. J. Ambient. Intell. Humaniz. Comput..

Darchini-Tabrizi, M., Roudgar, A., Entezari-Maleki, R., Sousa, L., 2025. Distributed deep reinforcement learning for independent task offloading in mobile edge computing. J. Netw. Comput. Appl. 240, 104211.

Deng, T., Yu, Z., Yuan, D., 2024. Task offloading optimization in mobile edge computing under uncertain processing cycles and intermittent communications. Comput. Netw. 245, 110359.

Diwaker, C., Singh, P., Jaiswal, R., Kumar, S., 2024. OCL-MEC: An online CPU-core prediction based on load in a mobile edge computing environment. Comput. Commun. 223, 27–38.

Dong, S., Tang, J., Abbas, K., Hou, R., Buyya, R., 2024a. Task offloading strategies for mobile edge computing: A survey. Comput. Netw. 254, 110791.

Dong, S., Tang, J., Abbas, K., Hou, R., Kamruzzaman, J., Rutkowski, L., Buyya, R., 2024b. Task offloading strategies for mobile edge computing: A survey. Comput. Netw. 254, 110791.

Esmaeili, M.E., Khonsari, A., Dolati, M., 2025. Dynamic distance-based load balancing in mobile edge computing with deep reinforcement learning. Comput. Commun. 108337.

He, X., Ni, Y., Tao, H., 2025. Lyapunov-based queue stability optimization for task offloading in UAV-assisted VEC. Pervasive Mob. Comput. 102126.

He, L., Sun, G., Sun, Z., Wang, P., Li, J., Liang, S., Niyato, D., 2024. An online joint optimization approach for QoE maximization in UAV-enabled mobile edge computing. In: IEEE INFOCOM 2024-IEEE Conference on Computer Communications. IEEE, pp. 101–110.

Hsieh, L.-T., Liu, H., Guo, Y., Gazda, R., 2023. Deep reinforcement learning-based task assignment for cooperative mobile edge computing. IEEE Trans. Mob. Comput. 23 (4), 3156–3171.

Huang, S., Wang, Z., Zhang, H., Wang, X., Zhang, C., Wang, W., 2023. One for all: Unified workload prediction for dynamic multi-tenant edge cloud platforms. In: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. KDD '23, Association for Computing Machinery, New York, NY, USA, pp. 788–797.

Kim, M., Jang, J., Choi, Y., Yang, H.J., 2024. Distributed task offloading and resource allocation for latency minimization in mobile edge computing networks. IEEE Trans. Mob. Comput..

Kong, Z.J., Xu, Q., Meng, J., Hu, Y.C., 2023. Accumo: Accuracy-centric multitask offloading in edge-assisted mobile augmented reality. In: Proceedings of the 29th Annual International Conference on Mobile Computing and Networking. pp. 1–16.

Kong, L., Xu, X., Ren, J., Zhang, W., Pan, L., Chen, K., Ooi, W.T., Liu, Z., 2025. Multi-modal data-efficient 3d scene understanding for autonomous driving. IEEE Trans. Pattern Anal. Mach. Intell..

Liang, Y., Zhang, S., Wu, J., 2025. Online optimization of offloading video analytics tasks to multiple edges for accuracy maximization. In: ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing. ICASSP, IEEE, pp. 1–5.

Lin, H., Yang, L., Guo, H., Cao, J., 2024. Decentralized task offloading in edge computing: An offline-to-online reinforcement learning approach. IEEE Trans. Comput. 73 (6), 1432–1445.

Madiyev, A., Cho, S., Kim, Y., Park, J., 2025. Energy-efficient offloading framework for mobile edge/cloud computing based on predictive analysis and DQN. J. Supercomput..

Nie, Y., Chai, Z., Lu, L., Li, Y., 2025a. Task offloading in edge computing: An evolutionary algorithm with multimodel online prediction. IEEE Internet Things J. 12 (3), 2347–2358.

Nie, Y., Chai, Z.-Y., Lu, L., Li, Y.-L., 2025b. Task offloading in edge computing: An evolutionary algorithm with multimodel online prediction. IEEE Internet Things J. 12, 2347–2358.

Qin, Y., Chen, J., Jin, L., Yao, R., Gong, Z., 2025. Task offloading optimization in mobile edge computing based on a deep reinforcement learning algorithm using density clustering and ensemble learning. Sci. Rep. 15 (1), 211.

Shustova, E., Sheshukova, M., Samsonov, S., Frolov, E., 2025. Scalable LinUCB: Low-rank design matrix updates for recommenders with large action spaces. arXiv preprint arXiv:2510.19349.

Siino, M., Giuliano, F., Tinnirello, I., 2025. Integrating large language models into network testbeds: A novel approach for automated experimentation and optimization. J. Netw. Comput. Appl. 243, 104281.

Song, L., Sun, G., Yu, H., Niyato, D., 2025. ESPD-LP: Edge service pre-deployment based on location prediction in MEC. IEEE Trans. Mob. Comput..

Su, Y., Zhao, Y., Niu, C., Liu, R., Sun, W., Pei, D., 2019. Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. KDD '19, Association for Computing Machinery, New York, NY, USA, pp. 2828–2837.

Tang, M., Wong, V.W., 2020. Deep reinforcement learning for task offloading in mobile edge computing systems. IEEE Trans. Mob. Comput. 21 (6), 1985–1997.

Tian, X., Yu, J., Luo, C., Yu, D., Wang, G., Feng, B., 2025. Robust dynamic broadcasting for multi-hop wireless networks under time-varying connectivity and dynamic SINR. IEEE Trans. Mob. Comput..

Wang, J., Hu, J., Min, G., Zomaya, A.Y., Georgalas, N., 2020. Fast adaptive task offloading in edge computing based on meta reinforcement learning. IEEE Trans. Parallel Distrib. Syst. 32 (1), 242–253.

Wang, L., Zhou, W., Zhai, L., 2025. Task offloading strategy of vehicle edge computing based on reinforcement learning. J. Netw. Comput. Appl. 239, 104195.

Wu, H., Hu, T., Liu, Y., Zhou, H., Wang, J., Long, M., 2022. Timesnet: Temporal 2d-variation modeling for general time series analysis. arXiv preprint arXiv:2210.02186.

Wu, H., Hu, T., Liu, Y., Zhou, H., Wang, J., Long, M., 2023. TimesNet: Temporal 2D-variation modeling for general time series analysis. In: International Conference on Learning Representations.

Wu, B., Ma, L., Ji, Y., Cong, J., Xu, M., Zhao, J., Yang, Y., 2025. Multi-layer guided reinforcement learning task offloading based on softmax policy in smart cities. Comput. Commun. 235, 108105.

Xie, B., Cui, H., 2025. Deep reinforcement learning-based dynamical task offloading for mobile edge computing. J. Supercomput. 81 (1), 35.

Xiu, Y., Gorlatova, M., 2025. Detecting visual information manipulation attacks in augmented reality: a multimodal semantic reasoning approach. IEEE Trans. Vis. Comput. Graphics.

Yang, Y., Chen, Y., Chen, N., Chen, J., 2025. ESFMTO: A reliable task offloading strategy based on edge server failure model in iIoT. Ad Hoc Networks 103887.

Zhong, X., Li, H., Li, L., Zhang, S., Liu, F., 2025. Communication-efficient task-offloading in mobile edge computing system: A multi-agent multi-armed bandit approach. IEEE Trans. Cloud Comput..

Zhou, J., Hou, X., Zeng, Y., Cong, P., Jiang, W., Guo, S., 2025. Quality of experience and reliability-aware task offloading and scheduling for multi-user mobile-edge computing systems. IEEE Trans. Serv. Comput. 18 (3), 1683–1696.

Zhu, L., Tan, L., Li, B., 2025. A vehicular edge computing offloading and task caching solution based on spatiotemporal prediction. Future Gener. Comput. Syst. 166, 107679.

Ting Li received the BS degree in communications engineering from Chongqing University, China, in 2018 and the Ph.D. degree in cyberspace security from the Institute of Information Engineering, Chinese Academy of Sciences, China, in 2023. Currently, she is an assistant professor in the School of Computer, Electronics and Information, Guangxi University, China. Her research interests revolve around the areas of Network Optimization, Mobile Edge Computing (MEC) and Privacy Protection.

Yinan Mi received the BS degree in Computer Science and Technology from North China University of Water Resources and Electric Power in Zhengzhou, China in 2024. Currently, he is pursuing a master's degree in Computer Technology at Guangxi University. His research interests include Mobile Edge Computing (MEC) and network optimization.

Kai Yang received his Ph.D. degree from the Institute of Information Engineering at the Chinese Academy of Sciences, China, in 2022. Currently, he is an assistant professor in the School of Computer, Electronics and Information, Guangxi University, China. His research interests revolve around Internet of Things and Network Security.